

Electronics & Robotics Club, BITS Goa

Week 2 Assignment

The Delivery Robot

Dynamic Programming on a Warehouse MDP

How This Assignment Works

Last week you solved a Markov Decision Process by hand with the Bellman equations. This week you turn those equations into algorithms and let them drive a robot. A delivery robot has to carry a package across a warehouse to the customer's door, dodging a hazard and coping with a slippery floor. You will write the four Dynamic Programming methods from the notes and use them to solve for the best route through progressively larger warehouses.

The two parts and the files

Part A (on paper). Four short questions on a tiny four-cell corridor. This is where you run one sweep of the algorithms by hand, with numbers small enough to check in your head. Handed in as a PDF.

Part B (in code). The real warehouses. You fill in the six functions in `dp.py`, check them with `python utils/check.py`, and draw your results with `python utils/run.py`.

When you unzip the assignment you get `dp.py` (the only file you edit), `assignment.pdf`, and this README, next to three folders: `figures/` (your plots and video appear here), `env/` (the environment and a setup checker), and `utils/` (the warehouse, the grader, and the plotting code, which you do not touch).

Local this week, no notebook. Everything runs on your own machine in the course `rl` environment (see the `setup/` folder if you have not built it). It behaves identically on Ubuntu and on WSL.

```
conda activate rl
python env/verify_env.py (checks your setup, and installs anything missing)
python utils/check.py (grades your dp.py)
```

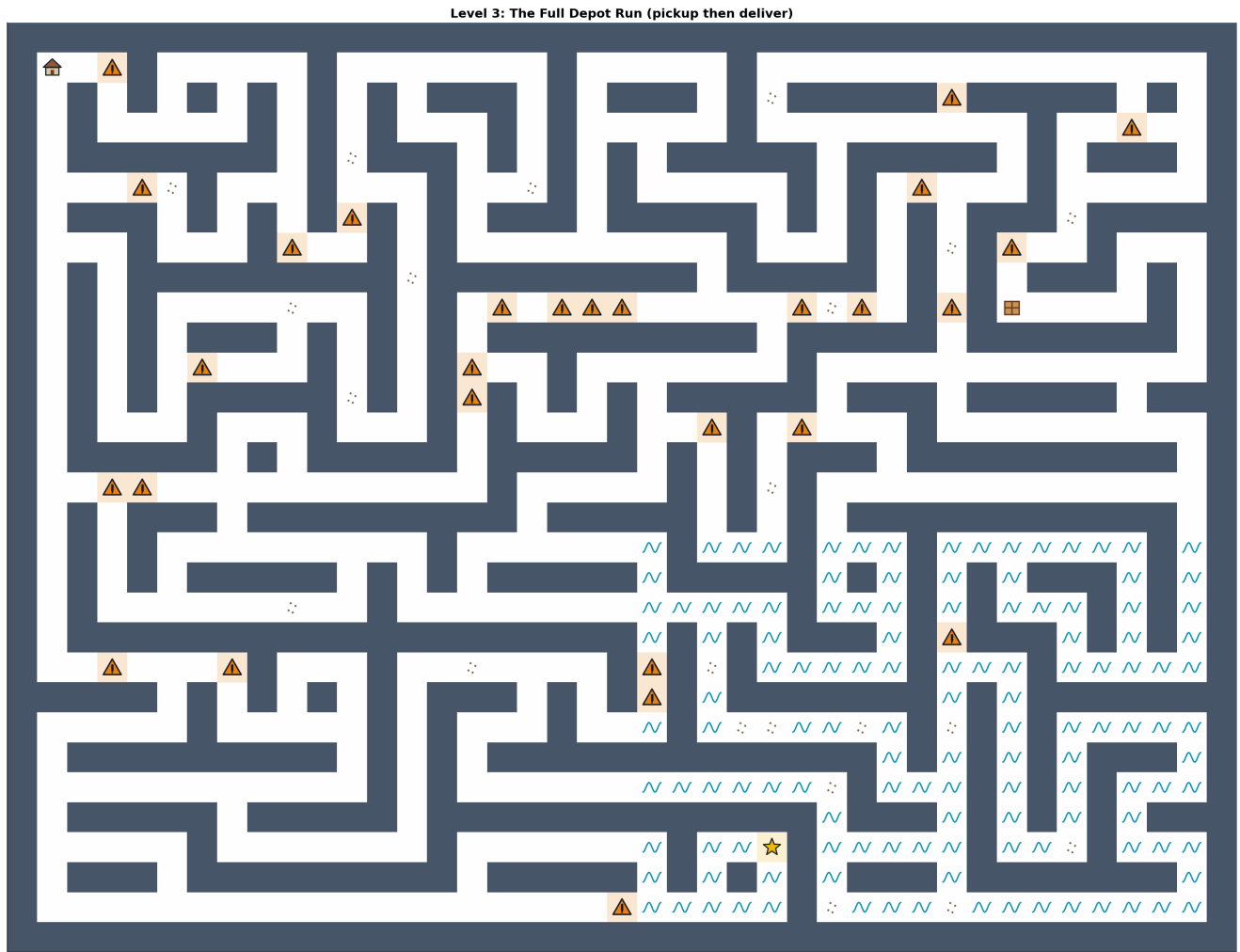
`check.py` grades each function against trusted answers and against a Monte Carlo simulation. A function you have not written yet is reported politely, not as a crash, so you can fill them in one at a time.

The loop for every task. Read the docstring in `dp.py` and the chapter of the notes it points to, work out the update for yourself, write the loop over states, then run `python utils/check.py`. When it is green, run `python utils/run.py` to save your figures and the rollout video into the `figures/` folder.

What you write. The six functions in `dp.py`: `policy_evaluation`, `greedy_policy`, `policy_iteration`, `value_iteration`, `value_iteration_inplace`, and `modified_policy_iteration`. That is the whole of Week 2, and it is the part worth your effort. The world and the plots are handed to you.

What to hand in (on Google Classroom).

- One **PDF** with your Part A answers (H1 to H4).
- Your completed **dp.py** with a passing `python utils/check.py`.
- Your Level 3 `figures/rollout.mp4`. You must upload this video when you hand in.



The Level 3 warehouse: a maze of more than two thousand states, with dozens of hazards (**warning triangles**), many rough tiles (**dotted**), and a large slippery region (**wavy lines**), with the pickup (the **parcel box**) and the door (the **gold star**) far apart. Your Dynamic Programming code solves it exactly in a fraction of a second.

Part A: Dynamic Programming by Hand

Work on this tiny corridor of four cells. Cell 3 is the door **G**; the other cells are floor. Moving costs -1 ; arriving at the door earns $+10$; the door ends the episode. Use $\gamma = 0.9$. Actions are `left` and `right`; at the ends, moving into the wall keeps you where you are (and still costs -1). (Reward design is a modelling choice: this corridor charges a small cost per step so the arithmetic is concrete, while the warehouses in Part B instead reward reaching the door and let the discount favour speed. The same machinery solves both.)

cell 0 (S)	cell 1	cell 2	cell 3 (G)
floor	floor	floor	door $+10$

- H1. By hand:** Take the policy “always go right” and start from $v_0(s) = 0$ everywhere. Do **one** sweep of iterative policy evaluation and write $v_1(0), v_1(1), v_1(2)$. Then do a second sweep for v_2 . (The door has value 0; it is terminal.)
- H2. By hand:** Suppose after several sweeps you have $V = (6.2, 8, 10, 0)$ for cells $(0, 1, 2, 3)$. At cell 1, compute the two action values $q(1, \text{right})$ and $q(1, \text{left})$, and say which action the greedy policy picks. This is one step of policy improvement.

- H3. By hand:** Now the floor is slippery. From cell 2 the action `right` reaches the door with probability 0.8 (reward +10) but slips and stays at cell 2 with probability 0.2 (reward -1). Using $V(2) = 10$, write the Bellman backup $q(2, \text{right})$ as a sum over the two outcomes and compute it.
- H4. By hand:** Consider the policy “always go left” evaluated at cell 0, and set $\gamma = 1$ (no discount). The robot bumps the left wall forever. What is $v(0)$? Now set $\gamma = 0.9$ and compute $v(0)$ for the same policy. Explain in one sentence why undiscounted Dynamic Programming needs a policy that actually reaches a terminal state.

Common Pitfall: a policy that never ends

At $\gamma = 1$, a policy that can loop forever without reaching the door has a value of $-\infty$, and policy evaluation never converges. This is why, at $\gamma = 1$, policy iteration must start from a policy that terminates. With $\gamma < 1$ (as in Part B) any policy has a finite value, so you are safe.

Part B: The Solver

Fill in the six functions in `dp.py`. Each is one equation from the notes turned into a loop. The helper `q_value` computes the one-step lookahead $q(s, a)$ for you; use it everywhere. Run `python utils/check.py` after each one.

Level 1: The Basic Delivery (deterministic)

In code: Write `policy_evaluation`, `greedy_policy`, `policy_iteration`, and `value_iteration`. Get all four green in `check.py`.

- **Prediction.** Evaluate the uniform-random policy (the robot wanders). Its value is poor everywhere: wandering is expensive.
- **Control.** Solve for the optimal policy with both policy iteration and value iteration. They reach the same answer. Note how many outer iterations policy iteration needs versus how many sweeps value iteration needs (`run.py` prints both and draws the race in `pi_vs_vi.png`).
- **Deliverable.** `level1_value_policy.png`: the optimal value of every cell with an arrow for the best move.

Level 2: The Slippery Floor (stochastic)

In code: Nothing new to write. Your same four functions now run on a slippery version of the warehouse, where each move slides sideways with probability p .

- The Bellman backup is now a real average over outcomes, not a single next state.
- `run.py` draws `slip_panel.png`: the optimal policy at $p = 0.00, 0.05, 0.15, 0.25$. **Answer in your report:** what happens to the value of the start as p rises, and what does the policy do differently near the hazard?

Level 3: The Full Depot Run (pickup then deliver)

Now the robot must fetch the package at the pickup (the **parcel box**) before a delivery at the door (the **gold star**) counts. The state is the pair (cell, carrying-the-package-or-not), so the table doubles. This warehouse is a maze of over two thousand states (the figure on the first page). Your code solves it exactly and reads off the whole plan, fetch then deliver, in a fraction of a second.

In code: Write the last two functions. `value_iteration_inplace` does the asynchronous, in-place updates; `modified_policy_iteration` is the k dial.

- **Asynchronous DP.** `value_iteration_inplace` writes each new value straight back into the same array. `run.py` draws `async_policy.png`, which shows that in-place updates settle the optimal policy in

fewer sweeps than the two-array version, and that sweeping the states nearest the door first is faster still. This is the idea behind prioritised sweeping in the notes.

- **The k dial.** `modified_policy_iteration` evaluates each policy with only k sweeps. `modified_pi.png` shows $k = 1$ reproducing value iteration and large k reproducing policy iteration.
- **Curse of dimensionality.** `curse.png` plots solve time against the number of states. Adding the one-bit “carrying the package” fact doubled the table. **Answer in your report:** this one extra bit already doubled the table. What happens to it if the robot also tracks fuel, the time of day, and three more packages?
- **Deliverable.** the two-panel `level3_value_policy.png` (before and after pickup) and `rollout.mp4`, a video of your robot running its optimal policy through the huge maze. Both land in `figures/`.

Watching and Debugging with the Video

Everything you look at lands in the `figures/` folder, next to `dp.py`; you never open `utils/`. To watch the run, on Ubuntu type `xdg-open figures/rollout.mp4`, and on WSL type `explorer.exe figures` and double-click `rollout.mp4`.

The video is a debugging tool, not just a demo. On the left, your robot walks the route your policy chose, one step at a time. On the right, a panel shows, for the cell it is on: the value V , the four action values $q(s, a)$ with the move your policy took and the move with the highest q both marked, the reward and running return, and a note when the slippery floor makes it slide. If your policy ever picks a move that is *not* the highest q , the panel says so, which usually means a bug in `greedy_policy`. So when your robot walks into a wall, drives into a hazard, or takes a silly detour, scrub to that step and read the panel to see exactly what your code computed there.

Optional: The Grand Warehouse

`maps.py` also holds a larger map, the Grand Warehouse. If you want more practice, solve it and look at how many value-iteration sweeps it takes, then try editing it, or writing your own map (any size, with your own walls, hazards, and slippery patches), and watch how your solver copes. This is for your own exploration; there is nothing to submit.